Introduction to machine learning
Exercise 1

Spring 2026

Submission guidelines, please read and follow carefully:

- The exercise **must** be submitted in pairs.
- Submit via Moodle.
- The use of LLMs is **strictly forbidden** unless otherwise stated.
- The submission should include two separate files:
  1. A pdf file that includes your answers to all the questions.
  2. The code files for the python question. You must submit a copy of the shell python file provided for this exercise in Moodle, with the required functions implemented by you. **Do not change the name of this file.** In addition, you can also submit other code files that are used by the shell file.
- Your python code should follow the course python guidelines. See the Moodle website for guidelines and python resources.
- Before you submit, **make sure that your code works in the course environment**, as explained in the guidelines. Specifically, **make sure that the test `simple_test` provided in the shell file works**.
- You may only use python modules that are explicitly allowed in the exercise or in the guidelines. If you are wondering whether you can use another module, ask a question in the exercise forum. No module containing machine learning algorithms will be allowed.
- For questions, use the exercise forum.
- Grading: Q.1 (python code): 20 points, Q.2: 20 points, Q.3: 20 points, Q.4: 20 points, Q.5: 20 points.
  - Unless otherwise specified, a question's points are evenly distributed among its subsections.

Question 1. Implement a function that runs the k-nearest-neighbors algorithm that we saw in class on a given training sample, and a second function that uses the output classifier of the first function to predict the label of test examples. We will use the Euclidean distance to measure the similarity between examples.

The shell Python file `nearest_neighbour.py` is provided for this exercise in Moodle. It contains empty implementations of the functions required below. You should implement them and submit according to the submission instructions.

The first function, `learnknn`, creates the classification rule. Implement the function in the file which can be found on the assignment page in the course's website. The signature of the function should be:

```
def learnknn(k, x_train, y_train)
```

The input parameters are:

- `k` - the number k to be used in the k-nearest-neighbor algorithm.
- `x_train` - a 2-D matrix of size $m \times d$ (rows $\times$ columns), where m is the sample size and d is the dimension of each example. Row i in this matrix is a vector with d coordinates which specifies example x_i from the training sample.
- `y_train` - a column vector of length m (that is, a matrix of size $m \times 1$). The i-th number in this vector is the label y_i from the training sample. You can assume that each label is an integer between 0 and 9.

The output of this function is `classifier`: a data structure that keeps all the information you need to apply the k-NN prediction rule to new examples. The internal format of this data structure is your choice.

The second function, `predictknn`, uses the classification rule that was outputted by `learnknn` to classify new examples. It should also be implemented in the `nearest_neighbour.py` file. The signature of the function should be:

```
def predictknn(classifier, x_test)
```

The input parameters are:

- `classifier` - the classifier to be used for prediction. Only classifiers that your own code generated using `learnknn` can be used here.
- `x_test` - a 2-D matrix of size $n \times d$, where n is the number of examples to test. Each row in this matrix is a vector with d coordinates that describes one example that the function needs to label.

The output is `y_testprediction`, which is a column vector of length n. Label i in this vector describes the label that `classifier` predicts for the example in row i of the matrix `x_test`.

Important notes:

- You may assume all the input parameters are legal.
- The Euclidean distance between two vectors z_1, z_2 of the same length can be calculated using `numpy.linalg.norm(z1-z2)` or `scipy.spatial.distance.euclidean(z1, z2)`.

Example for using the functions (here there are only two labels, 0 and 1):

```
>>> from nearest_neighbour import learnknn, predictknn
>>> k = 1
>>> x_train = np.array([[1,2], [3,4], [5,6]])
>>> y_train = np.array([1, 0, 1])
>>> classifier = learnknn(k, x_train, y_train)
>>> x_test = np.array([[10,11], [3.1,4.2], [2.9,4.2], [5,6]])
>>> y_testprediction = predictknn(classifier, x_test)
>>> y_testprediction
[1, 0, 0, 1]
```

Question 2. **Note:** For the following experiments, you are permitted to use LLMs to assist in writing the scripts. However, it is your responsibility to ensure that the LLM's implementation is correct and efficient, as inefficient code may lead to excessive runtimes during the required simulations.

Test your k-nearest-neighbor implementation on the hand-written digits recognition learning problem: In this problem, the examples are images of hand-written digits, and the labels indicate which digit is written in the image. The full dataset of images, called MNIST, is free on the web. It includes 70,000 images with the digits 0-9. A numpy format file that you can use, called `mnist_all.npz`, can be found on the assignment page in the course website. For this exercise, we will use a smaller dataset taken out of MNIST, that only includes images with the digits 1, 3, 4, and 6, so that there are only four possible labels. Each image in MNIST has 28×28 pixels, and each pixel has a value indicating how dark it is. Each example is described by a vector listing the $28 \cdot 28 = 784$ pixel values, so we have $X \in \mathbb{R}^{784}$: every example is described by a 784-coordinate vector.

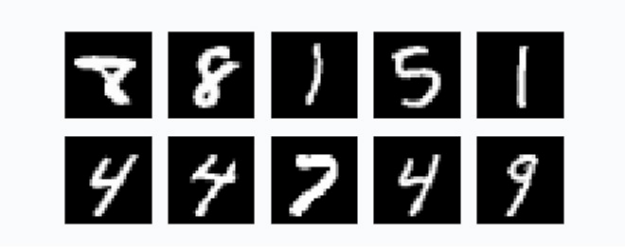

Figure 1: Some examples of images of digits from the dataset MNIST

The images in MNIST are split into training images and test images. The test images are used to estimate the success of the learning algorithm: In addition to the training sample S as we saw in class, we have a test sample T of size n, which is also a set of labeled examples.
The k-NN algorithm gets S, and decides on $\widehat{h}_S$. Then, the prediction function can predict the labels of the test images in T using $\widehat{h}_S$. The error of the prediction rule $\widehat{h}_S$ on the test images is:

$$\mathcal{L}_T(\widehat{h}_S) = \frac{1}{n} \sum_{(x,y)\in T} \mathbb{1}\{\widehat{h}_S(x) \neq y\}.$$

Since T is an i.i.d. sample from $\mathcal{D}$, $T \sim D^n$, the error on T is a good estimate of the error of $\widehat{h}_S$ on the distribution $\mathcal{L}_{\mathcal{D}}(\widehat{h}_S)$.

To load all the MNIST data, run the following command (after making sure that the `mnist_all.npz` file is in your working directory):

```
>>> data = np.load('mnist_all.npz')
```

This command will load the MNIST data to a Python dictionary called `data`. You can access the data by referencing the dictionary: `data['train0']`, `data['train1']`, ..., `data['train9']` `data['test0']`, `data['test1']`, ..., `data['test9']`.

To generate a training sample of size m with images only of some of the digits, you can use the function `gensmallm` which is provided in the shell file `nearest_neighbour.py`. The function is used as follows:

```
>>> (X, y) = gensmallm([labelAsample, labelBsample], [A, B], samplesize)
```

The function `gensmallm` selects a random subset from the provided data of labels A and B and mixes them together in a random order, as well as creates the correct labels for them. This can be used to generate the training sample and the test sample.

Answer the following questions in the file "answers.pdf":

(a) (2 points) Run your k-NN implementation with $k = 1$, on several training sample sizes between 1 and 100 (select the values of the training sizes that will make the graph most informative). For each sample size that you try, calculate the error on the full test sample. You can calculate this error, for instance, with the command:

```
>>> np.mean(y_test != y_testpredict)
```

Repeat each sample size 10 times, each time with a different random training sample, and average the 10 error values you got. Submit a plot of the average test error (between 0 and 1) as a function of the training sample size. Don't forget to label the axes. Present also error bars, which show what is the minimal and maximal error value that you got for each sample size. You can use Matlab's plot command (or any other plotting software).

(b) (2 points) Do you observe a trend in the average error reported in the graph? What is it? How would you explain it?

(c) (2 points) Does the size of the error bars change with the sample size? What trend do you see? What do you think is the reason for this trend?

(d) (4 points) Run your k-NN implementation for each of the following training sample sizes $m \in \{50, 100, 400\}$. For each sample size m,

- run for all odd values of k between 1 and 15;
- create and submit a separate plot of the test errors as a function of k;
  for each k, the reported test error is the average over 25 runs (each with a new randomly chosen sample of m examples).

What is the optimal value of k you got for each of the sample sizes? Explain the trend in each graph and between the graphs of different m values.

(e) (5 points) Check what happens when the labels are corrupted at different noise levels. Repeat the experiment for each sample size $m \in \{50, 100, 400\}$, but this time consider label noise levels of 0%, 5%, 10%, 15%, 20%, 25%, and 30%. For each training set and test set that you feed into your functions, first select a random subset of the examples according to the required noise level, and change each selected label to a different label, chosen uniformly at random from the other three possible labels.

For each sample size m, run your implementation only for $k \in \{1, 7, 15\}$, and create a plot of the test error as a function of the noise level. For each fixed sample size m, the three curves corresponding to $k = 1, 7, 15$ should appear on the same plot. Thus, you should create a total of three figures, one for each sample size $m \in \{50, 100, 400\}$. Now for each combination of m, k, and noise level, the reported test error should be the average over only 10 runs. In addition, add to each plot the Bayes-optimal error as a function of the noise level.

(f) (5 points) Compare the performance of the algorithm for the different values of k as the noise level increases. For each sample size m, discuss which value of k performs best at each noise level, and how the gap between the performances for $k = 1, 7, 15$ changes as the noise increases.

Relate your observations to the intuition and guarantees for nearest-neighbor-based learning under the assumptions discussed in class. What did you expect to happen as the noise level increases, and why? Do your experimental results match this expectation? If yes, explain why. If not, suggest possible reasons.

Question 3. Consider a distribution over rabbits and food preferences. Each rabbit likes either carrot or lettuce. For each rabbit, we measure their weight in kg and their age in months. In principle, a rabbit can live until age 60 months and weigh as much as 5 kg.

(a) (3 points) What should $\mathcal{X}$ and $\mathcal{Y}$ be in this problem? Define $\mathcal{X}$ to be as specific as possible given the problem description.

(b) (5 points) We have a distribution $\mathcal{D}$ over rabbits with the following probabilities (all other values have zero probability):

age (months)	weight (kg)	preferred food	probability
8	1	carrot	0%
8	1	lettuce	15%
8	3	carrot	0%
8	3	lettuce	47%
14	1	carrot	18%
14	1	lettuce	0%
14	3	carrot	0%
14	3	lettuce	20%

Denote the Bayes-optimal predictor for $\mathcal{D}$ by h_{bayes}. Write the value of $h_{\text{bayes}}(x)$ for each x in the support of $\mathcal{D}$. What is the Bayes-optimal error of $\mathcal{D}$?

(c) (6 points) Suppose we now only measure the age of the rabbits, and so we have a distribution $\mathcal{D}'$ which is exactly like $\mathcal{D}$ except that the weight of the rabbit is not measured, only the age. Provide a table of probabilities for this distribution.

(d) (6 points) Denote the Bayes-optimal predictor for $\mathcal{D}'$ by h'_{bayes}. Write the value of $h'_{\text{bayes}}(x)$ for each x in the support of $\mathcal{D}'$. What is the Bayes-optimal error of $\mathcal{D}'$?

Question 4. Define a 'bad' distribution as a distribution satisfying $y = +1$ for all $x \in \mathcal{X}$, and denote the family of bad distributions by $\mathfrak{D}_{\text{bad}}$. If $\mathcal{D} \notin \mathfrak{D}_{\text{bad}}$, then we say that it is a 'good' distribution, and we denote the family of good distributions by $\mathfrak{D}_{\text{good}}$.

Suppose that $\mathcal{H}$ is a binary hypothesis class containing the constant hypothesis

$$h_{+1}(x) = +1 \quad \text{for all } x \in \mathcal{X},$$

and having the property that there exists an efficient learning algorithm A which is "almost" a PAC-learner for $\mathcal{H}$ in the following sense:

> For all $\varepsilon, \delta > 0$ and any distribution $\mathcal{D} \in \mathfrak{D}_{\text{good}}$, given $m(\varepsilon, \delta)$ i.i.d. examples from $\mathcal{D}$, with probability at least $1 - \delta$, the algorithm returns a predictor $\hat{h} : \mathcal{X} \to \{-1, +1\}$ such that
> $$\mathcal{L}_{\mathcal{D}}(\hat{h}) \leq \inf_{h \in \mathcal{H}} \mathcal{L}_{\mathcal{D}}(h) + \varepsilon.$$

Namely, A is a PAC-learner for $\mathcal{H}$, except on bad distributions (in which case no guarantee can be given on $\hat{h}$).

(a) (3 points) Suppose that $\mathcal{D} \in \mathfrak{D}_{\text{good}}$. Prove that if $p := \mathbb{P}_{(x,y)\sim\mathcal{D}}[y = -1]$, then the constant hypothesis $h_{+1} \equiv +1$ obtains risk $\mathcal{L}_{\mathcal{D}}(h_{+1}) = p$.

(b) (5 points) Given $\varepsilon > 0$, show that if $\mathcal{D} \in \mathfrak{D}_{\text{good}}$ is such that $p > \varepsilon$, then

$$\mathop{\mathbb{P}}_{S \sim \mathcal{D}^m} [y_i = +1 \text{ for all } i \in \{1, \ldots, m\}] \leq \exp(-\varepsilon m).$$

Namely, a good distribution will disclose that it is good with high probability if m is sufficiently large.

(c) (12 points) Devise an efficient PAC-learner A' for $\mathcal{H}$ which uses A, and use the previous two parts to prove its correctness. Analyze its sample complexity in terms of ε, δ and the sample complexity of A, and its running time in terms of the running time of A.
Hint: Split your analysis into two cases: When $\mathcal{D}$ is such that $p > \varepsilon$, and when $p \leq \varepsilon$.

Question 5. (20 points) Consider a binary classification problem with input space $\mathcal{X} = \mathbb{R}^2$ and label space $\mathcal{Y} = \{-1, +1\}$.

Recall that for binary classification we define

$$\text{sign}(x) = \begin{cases} +1, & x > 0, \\ -1, & x \leq 0. \end{cases}$$

(In particular, $\text{sign}(0) = -1$.)

Define the class of one-coordinate threshold hypotheses on $\mathcal{X} = \mathbb{R}^2$ and $\mathcal{Y} = \{-1, +1\}$ by

$$\mathcal{H}_{\text{thr}} \coloneqq \left\{ h_{i,b} : \mathbb{R}^2 \to \{-1, +1\} \;\middle|\; i \in \{1, 2\},\ b \in \mathbb{R},\ h_{i,b}(x) \coloneqq \text{sign}(x_i - b) \right\}.$$

What is the VC dimension of $\mathcal{H}_{\text{thr}}$? Prove your answer in detail.